RYO-CHAN Hackathon 2026 — Project Submission Form

Filled against the organiser's own template, downloaded from `https://ryobuild.com/RYOCHAN-Hackthon-Project-Submission-Form.pdf` on 2026-09-07 and committed unchanged beside this file as `RYOCHAN-Hackathon-Project-Submission-Form-BLANK.pdf`. That PDF has no interactive fields, so this is the same form filled in, in the same order, and rendered to `project-submission-form.pdf` by `scripts/submission_pdf.py`.

Members

Total Members		**1**
Name	**Role**	**Email**
Pugar Huda Mantoro	Sole builder: architecture, backend, agents, interface, tests	hudapugar@gmail.com

Discord: Lynx (hajislamet) · GitHub: PugarHuda

Overview

Field	**Value**
Project	Nota
Track	Track 1 (Autonomous Agents), Track 2 (Dashboards & Interfaces) and Track 3 (New Skills). Each is judged independently.
Problem	An AI opinion about a market is unfalsifiable. You cannot tell whether the number in a sentence came from the evidence or from the model, whether a missing feed was quietly treated as zero, or whether the answer would be the same tomorrow. Nothing in a chat log can be re-run, so nothing in it can be audited, and a confident paragraph is indistinguishable from a careful one.
Solution	Nota turns each decision into a receipt that reproduces. Three specialised agents argue over live RYO evidence and cite dotted paths into it; a judge weighs them by each agent's own Brier score from past calls; risk sizing is pure ATR arithmetic with no model in it. `nota replay <id>` rebuilds the decision from the stored evidence and the model output cached beside it, keyed by the model that produced them, and prints `identical: True`. That runs with no API key and no network, so anyone who clones the repository can check the claim. Nota also audits RYO itself: it recomputes RYO's RSI and ATR from public candles with Wilder's method, checks RYO's price against three exchanges, and compares its own sizing against RYO's published trade plan. When the sources disagree, the judge refuses to size a trade at all.
Key Features	Council of three agents with dotted-path citations, and citations pointing at absent evidence are dropped in code rather than trusted. Judge weighted by historical Brier score. Pure-ATR sizing that blocks rather than guesses. Replayable receipts, verifiable from the browser or the CLI. Diff-first dashboard that ranks what changed against the previous receipt by impact. Four research skills in RYO's own envelope, served both on RYO's `/api/skills` paths and as a real MCP server at `POST /mcp` with tools and resources. `nota watch` autonomous loop with Telegram and Discord publishing. Public backing on receipts, scored when the call resolves. Open Graph receipt cards, markdown and JSON exports, `llms.txt`, keyboard-first accessible interface in light and dark.
Target Users	Traders and analysts who need to justify a call rather than only make one; RYO itself, which can mount the four skills as routes or call them over MCP; and hackathon judges, who can verify every claim on this page from outside the project.

Field	Value
Scope	Read-only research over RYO's six builder tools plus independent public sources. Practice trades only: sizes, stops and targets are recorded and marked against later prices, and no order is ever placed, signed or routed. Ships with a ledger snapshot of four receipts made on live RYO evidence on 2026-09-07.
Limitations	Stated plainly rather than hidden. A council decision needs one LLM key; live RYO evidence needs the builder key; verification and the skills need neither. X's public syndication endpoint throttles data-centre addresses hardest, so on the hosted deployment `x:` voices usually report `unavailable` while `tg:` and `bs:` answer. Brier weights only begin to move once calls reach their seven-day horizon, so on a ledger this young every agent still carries weight 1.0 and the dashboard says so. The hosted deployment is read-only because a serverless filesystem cannot be written, so backing answers 503 there with an explanation. The receipt diff ranks changes with a documented heuristic, not a model.

Tech Stack

Field	Value
Frontend Stack	No framework and no build step: two hand-written pages of semantic HTML with native CSS and vanilla JavaScript, served by the same FastAPI process. One palette expressed with CSS `light-dark()` so light, dark and system all work from one definition, `prefers-reduced-motion` honoured, keyboard navigation and visible focus throughout. The hero mark is an inline SVG generated from the receipt's own evidence hash.
Backend Stack	Python 3.12, FastAPI and uvicorn, Pydantic v2 for every boundary type, httpx for outbound calls, SQLite in WAL mode as the ledger (evidence packs, LLM output cache, decisions, outcomes, backings), Typer for the `nota` CLI, Pillow for receipt cards, defusedxml for RSS, vaderSentiment for lexicon scoring. Deployed on Vercel as a framework-detected FastAPI app.
AI Model(s)	Provider-agnostic by design. Two adapters: Anthropic, and any OpenAI-compatible endpoint. The shipped receipts were produced through Venice with `qwen3-235b-a22b-instruct-2507`, and one older receipt was produced with `anthropic/claude-haiku-4.5`, which is why replay is keyed by the model that produced a receipt rather than by whatever is configured now. Model outputs are cached by `(evidence hash, role, prompt version, model)`.
Other Technologies	Model Context Protocol in both directions: Nota is an MCP client of RYO (JSON-RPC `tools/call`, selectable with `RYO_TRANSPORT=mcp`) and an MCP server of its own over Streamable HTTP at `POST /mcp`, negotiating protocol versions 2026-07-28 through 2024-11-05 and serving both `tools/*` and `resources/*`. Public data sources, all keyless: CoinGecko, Coinbase, Kraken, alternative.me Fear & Greed, t.me/s channel previews, the Bluesky public AppView, X's public syndication endpoint, and CoinDesk, Cointelegraph, The Block and Decrypt RSS. Tavily or Venice web search when a key is present. Playwright drives the browser QA suite and records the walkthrough video.

Repository / Demo

Field	Value
Github Repository	https://github.com/RYO-Digital/ryochan-hackathon_repository-235 (the repository issued for this project; all final code is on `main`). Public mirror of the same history: https://github.com/PugarHuda/nota
Demo Video	https://nota-ryo.vercel.app/demo.mp4 — a two-minute captioned walkthrough, served by the project itself, public and with no password. It is also committed to the repository at `nota/static/demo.mp4`.

Field	Value
Documentation	`README.md` in the repository is the primary document: how a decision is made, the honesty rules the code enforces, the MCP server, the dashboard, the skills, and how to evaluate everything with zero keys. `docs/skills/SKILL-SPEC.md` is the skill contract; `docs/HACKATHON-ANALYSIS.md` records what was verified about the platform; `docs/SUBMISSION.md` tracks this submission. Live: https://nota-ryo.vercel.app and https://nota-ryo.vercel.app/llms.txt

Testing Information

Field	Value
How to run the Project	Clone the repository, `uv sync`, then `NOTA_DB=data/demo.db uv run nota serve` and open http://127.0.0.1:8000 for the overview or http://127.0.0.1:8000/app for the dashboard itself. That serves the shipped ledger of four receipts made on live RYO evidence, so the dashboard, the replay verification, the receipt cards, the exports and the skill runner all work before any key is set.
Prerequisites	Python 3.12 or newer and uv. No database server, no Node, no Docker. For the browser tests only: `uv run playwright install chromium`.
Installation Steps	1. `git clone <repository> && cd nota` 2. `uv sync` 3. `cp .env.example .env` 4. optionally fill in the keys below 5. `uv run nota health` to confirm RYO is reachable.
Environment Variables	All optional for evaluation; `.env.example` lists every one with comments and no values. `RYO_MCP_KEY` for live RYO evidence, with `RYO_MCP_URL` and `RYO_TRANSPORT` (`rest` or `mcp`). One LLM key for the council: `ANTHROPIC_API_KEY`, or `NOTA_LLM=openai` with `OPENAI_BASE_URL` and `OPENAI_API_KEY` for any OpenAI-compatible provider, plus `NOTA_MODEL` and `NOTA_MAX_TOKENS`. `NOTA_DB` selects the ledger, `NOTA_READONLY=1` opens it immutably, `NOTA_PUBLIC_URL` sets the base for permalinks. Optional: `NOTA_VOICES`, `TAVILY_API_KEY`, `TELEGRAM_BOT_TOKEN`, `TELEGRAM_CHAT_ID`, `DISCORD_WEBHOOK_URL`. No real key is committed anywhere, and the git history was scanned to prove it.
Build Command	None. There is no build step: no bundler, no transpiler, no generated assets. `uv sync` installs the locked dependencies from `uv.lock` and the app runs from source.
Run Command	`uv run nota serve` for the interface and the API, or `uv run nota decide SOL` for one decision, `uv run nota watch SOL,BTC --every 3600 --notify` for the autonomous loop, `uv run nota replay <id>` to verify a receipt, `uv run nota skill run price_crosscheck '{"symbol":"SOL"}'` for a skill. `uv run nota --help` lists all twelve commands plus the `skill` group (`skill spec`, `skill run`).
Test Command	`uv run pytest -q` — 175 tests, currently all passing. It needs no network and no keys: unit tests, API tests, MCP server and transport tests, and thirteen browser QA checks that drive a real uvicorn with Playwright over the shipped ledger, failing on any console error or failed request. Without Chromium installed the browser checks skip with a clear reason rather than failing.
Test Account(s)	None needed. Nothing in Nota has a login: there are no accounts, no sessions and no wallets. Backing a receipt asks only for a handle, which is a label rather than an identity, and the hosted deployment is read-only so it refuses backing with a 503 that explains why.

Declarations

- **No secret has ever been committed**, and the organiser's review of git history was anticipated: 49 commits and 448 blobs scanned against twelve credential patterns (RYO builder keys, Anthropic,

OpenAI, OpenRouter, Venice, Tavily, Telegram bot tokens, Discord webhooks, AWS, GitHub and Vercel tokens, PEM private keys). The single match is `ryo_mcp_your_private_key`, the placeholder inside RYO's own builder guide that this repository quotes at `docs/MCP-Builder-Guide.md`. No `.env` or credential file was added in any commit; `.env.example` carries names only.

- **No fabricated data is presented as real.** Every receipt prints its own source label. `live` means RYO answered; `recorded` means a real RYO response captured with `nota record` and replayed; `fixture` means a hand-built test fixture. `data_mode` is carried through from RYO per section, a value that could not be fetched stays `null` and is never turned into zero, and the risk layer refuses to size a trade on evidence RYO marks `simulated`.
- **Third-party code and resources are disclosed** in the README. No starter template was used; the repository began empty and every line of application code was written during the hackathon. Libraries: httpx, pydantic, fastapi, uvicorn, typer, python-dotenv, anthropic, vaderSentiment, defusedxml, pillow; development only: pytest, respx, playwright. The `taste-skill` design ruleset was read while reshaping the interface and is gitignored rather than vendored, so no file of it ships here.
- **Read-only.** The RYO surface used is read-only, and Nota never places, signs or routes an order. Practice positions exist only in the local ledger and are labelled as practice everywhere they appear.